Stress-Testing SynthID:
A Hands-On Comparison of GAN-Based and Regeneration-Based Attacks on AI-Generated Image Watermarks

Tahseen Rabbani[1] and Thomas Mason[2]

[1]Department of Computer Science, University of Chicago
[2]Coalfire Systems

June 15, 2026

Abstract

SynthID [Google DeepMind, 2023] is Google DeepMind's invisible image watermarking scheme deployed on the outputs of Gemini's image generators (Nano-Banana / Gemini 2.5 Flash Image). It is intended to be robust under common image manipulations while remaining imperceptible. We probe its robustness through the *diffusive-regeneration* attack of Zhao et al. [Zhao et al., 2023] as adapted by the WAVES benchmark [An et al., 2024], which round-trips a watermarked image through a publicly available Stable Diffusion model at varying noising depths. To score the attack at scale without exhausting Google AI Studio quota, we train a ResNet-18-based binary classifier on Apple's Pico-Banana-400K dataset [Qian et al., 2025] as a SynthID-shaped surrogate detector, and use it as a fast iteration target while spot-checking attacked images against the deployed SynthID detector for ground truth. On a held-out set of 104 Nano-Banana-generated, SynthID-watermarked images the surrogate detector reaches 91.3% NB-detection rate on unattacked input and drops monotonically as the regen attack depth N grows: 64.4% at N=10, 58.7% at N=20, 53.8% at N=40, 45.2% at N=80. The deployed SynthID detector tracks the surrogate's response qualitatively. The full code, the test set, the pre-computed regen attack outputs at N=10, 20, 40, 80, the generation prompts, and the detector training pipeline are released as a hands-on kit at `https://github.com/rabbanitw/WAVES/tree/synthid-regen-kit`.

1 Introduction

Generative image models from Google, OpenAI, and others now ship with *invisible watermarks* that are embedded at generation time and intended to survive routine image manipulation. The most-deployed example is Google DeepMind's SynthID [Google DeepMind, 2023], attached to every image produced by the Gemini image-generation family (Nano-Banana / Gemini 2.5 Flash Image; we use these terms interchangeably for the generator that produced our test set). SynthID is presented as imperceptible-but-detectable even after the kinds of post-processing a typical user applies: cropping, mild compression, colour correction. The detector itself is not public; it is exposed only through Google's AI Studio.

This work targets the natural follow-up question: *how brittle is SynthID against attackers who do not have decoder access*? Our methodology combines two pieces:

Diffusive-regeneration attack (Section 3). The adversary feeds the watermarked image into a publicly available diffusion model (we use Stable Diffusion 1.4 [Rombach et al., 2022]), forward-noises the latent for N scheduler steps, and reverse-denoises back to pixel space. The diffusion

model has no knowledge of SynthID's embedding mechanism, but the round-trip non-trivially perturbs the high-frequency structure that any spread-spectrum watermark needs to ride on. Zhao et al. introduced this attack in 2023 [Zhao et al., 2023]; WAVES [An et al., 2024] adopted it as their primary benchmarked attack under the name *Regen-Diff* and reported it as the most damaging single-stage attack against three different academic watermarks (Tree-Ring, Stable-Signature, Stega-Stamp). The question this attack answers is: does the same attack mechanism continue to work against the production SynthID stack?

Surrogate detector (Section 2). The deployed SynthID detector is exposed only via Google AI Studio: slow per query, rate-limited, no API for bulk evaluation, no gradient access. We therefore train a SynthID-shaped surrogate: an ImageNet-pretrained ResNet-18 [He et al., 2016] fine-tuned to distinguish natural photographs from Nano-Banana edits on Apple's Pico-Banana-400K dataset [Qian et al., 2025]. The surrogate is *not* a faithful reverse-engineering of SynthID's payload; it is a learned "looks like a Nano-Banana output" detector that captures NB-output statistics (stylistic / micro-textural / watermark-carrier-band) which we use as a fast iteration target for the regeneration attack. We then qualitatively verify the surrogate's response tracks the deployed SynthID detector under regen.

Our contribution is a reproducible empirical pipeline that releases (a) a held-out 104-image SynthID test set with its generation prompts, (b) a clean implementation of the symmetric-DDIM regen attack of Zhao et al., (c) pre-computed regen outputs at $N \in \{10, 20, 40, 80\}$ for every test image, and (d) the surrogate detector's training and evaluation pipeline. The numbers line up with what the WAVES paper predicts at this attack scale: the surrogate detector flags 91.3% of unattacked Nano-Banana outputs as NB and drops monotonically as N grows, reaching 45.2% at N=80, with spot-checks against the deployed SynthID detector confirming the directionality. The released kit targets the DEFCON 34 AI Village.

Test set. We curated a held-out set of **104** SynthID-watermarked images (512×512 JPEG, generated by Nano-Banana via the Gemini 2.5 Flash Image API). The generation prompts (one per image, indexed 0–103) were sourced from a list of 150 short "camera+lens+subject" phrases of the form: "*¡style prefix¿*: Photo shot on camera: *¡subject description¿*" (e.g., *"Disposable camera flash photo, 1990s style: Photo shot on camera: side view of a brown horse wearing a saddle. The number 55 is stamped in white on its rear flank."*). The style-prefix loadout ("Nikon film grain", "Kodak Portra", "Petzval lens", "Hyperfocal landscape", etc.) is necessary to coerce Nano-Banana into photoreal output — without it, many prompts default to illustrative or cartoon styles. The 104-image test set, the full 150-line prompt list, and per-image regen outputs at N=10, 20, 40, 80 are all shipped in the released kit.

2 A SynthID-Shaped Surrogate Detector

To score the regeneration attack at scale we need a callable detector that approximates SynthID's behaviour. The deployed SynthID detector itself is exposed only through Google AI Studio — it is slow per query, rate-limited, and not gradient-accessible — so as a stand-in we train a binary classifier C on (real photograph, Nano-Banana edit) pairs. C is not a faithful reverse engineering of SynthID's embedding; it is a learned "does this look like a Nano-Banana output" detector that captures the stylistic, micro-textural, and watermark-carrier-band statistics that distinguish NB outputs from natural photography. We will report below that its responses *do*, in practice, track

the deployed detector's responses under distortion-based attacks, which is what makes it useful as a fast local evaluator.

2.1 Training data

We train C on Apple's Pico-Banana-400K dataset [Qian et al., 2025]. Pico-Banana-400K supplies ∼258K (real Open Images photo, Nano-Banana edit) triples in its SFT split, each tagged with one of 35 *edit_type* labels. We filter to the ∼174K *photoreal* edit types — dropping the ∼30% of the dataset tagged with stylized categories like "*Van Gogh style*", "*cartoonify*", "*LEGO minifigure*", etc., because those edits make the NB-side image trivially distinguishable from a photograph on stylistic grounds and would teach the classifier a shortcut that has nothing to do with the watermark or NB's photoreal-output fingerprint. From the photoreal subset we sample a 3,447-pair working corpus partitioned as **train**=2,947, **val**=200, **test**=300. The 104-image SynthID test set from Section 1 serves as a *cross-pipeline* OOD evaluation: C has only seen Nano-Banana as an instruction-edit generator during training, and the SynthID test images come from a text-to-image prompt distribution.

2.2 Codec-matched preprocessing

The original-class images come from Open Images and are stored as JPEG; the NB-edited images come from Apple's CDN and are stored as PNG. A naive classifier picks up this codec mismatch and learns to discriminate JPEG-artefact signatures from PNG cleanness in well under one epoch. We therefore re-encode both classes through JPEG quality 95 in memory before either class touches the network. All metrics in this paper are with codec matching enabled.

2.3 Architecture and optimisation

C is a torchvision `resnet18` pretrained on ImageNet (`IMAGENET1K_V1` weights) [He et al., 2016] with the original 1000-way `fc` head replaced by a 2-way `nn.Linear(512, 2)`. Inputs are resized to 256×256 and ImageNet-normalised internally (registered as forward-time buffers so the rest of the kit's data pipeline can hand the model $[-1, 1]$ tensors directly). Total trainable parameters: 11.2M.

Training is 20 epochs at batch 64 using AdamW [Loshchilov and Hutter, 2019] (`lr=1e-4`, `weight_decay=1e-4`) with a cosine LR schedule over the full number of optimiser steps. We use a single A100-40GB at fp32; per-epoch wall clock is approximately 95 seconds and dominated by the in-loader JPEG re-encode rather than the model compute.

Software stack. PyTorch 2.1.0 on CUDA 11.8, torchvision 0.16.0, Pillow ≥ 10, numpy 1.26.4.

2.4 Detection metrics

The detector reaches 88.5% accuracy in-distribution and *also* flags 91.3% of the held-out SynthID images as Nano-Banana, despite the SynthID images coming from a different prompt distribution (text-to-image vs. image-edit) than its training data. The classifier is highly confident on the SynthID set — median p(NB)=1.000 — which is the empirical signature of a classifier that has locked onto something more durable than a dataset-specific artefact. The remaining 8.7% of the SynthID set that the detector misses are qualitatively the most "photographic-looking" images in the set; discriminator failures are concentrated on the easy-cases-for-the-attacker end of the test set.

Table 1: Detector metrics. The first three rows are in-distribution metrics on the Pico-Banana-400K held-out test split (600 single-image examples; 300 real photos with ground-truth label 0 and 300 NB edits with ground-truth label 1). The fourth row is the cross-pipeline metric on the unattacked SynthID test set (Section 1); "predicted as label 1" is the NB-detection rate.

metric	rate
overall test accuracy (Pico-Banana, balanced 600-example set)	**88.5**%
recall on real photographs (label 0)	83.0%
recall on NB edits (label 1)	**94.0**%
SynthID test set (104 images), predicted as label 1	**91.3**%
mean confidence p(NB)	0.914
median confidence p(NB)	1.000
# high-confidence (p>0.9)	91/104

Cross-pipeline transfer as a usability claim. We do *not* claim that C has learned the SynthID payload. What we do claim is that, on the unattacked side, C's detection rate on the deployed SynthID test set (91.3%) is high enough that the detector is informative as a fast iteration target: a regen attack that drops C's rate is plausibly disrupting the underlying NB-output fingerprint SynthID rides on, and we verify the qualitative directionality of that claim by spot-checking attacked images against Google AI Studio's deployed detector (Section 3). The deployed detector is the ground truth; C is a fast proxy that correlates well enough to drive attack development without spending Google's quota.

3 Diffusive Regeneration Attack

We follow the symmetric-DDIM regen recipe of Zhao et al. [Zhao et al., 2023], as used in WAVES Section F.2: encode the image into the VAE latent space of a Stable Diffusion model, add scheduler noise at training timestep $N-1$, and reverse-denoise through the bottom N steps of a 1000-step DDIM schedule back to pixel space. Concretely we use the SD-1.4 checkpoint [Rombach et al., 2022] at fp16, the diffusers DDIM scheduler, classifier-free guidance scale 7.5, and an empty text prompt. Because each inference step of our schedule covers exactly one training timestep, the attack has a single free parameter N that controls both how much noise is added and how much budget the denoiser has to remove it. This is in contrast to WAVES's published asymmetric variant (50-step inference schedule, $\sim N/20$ denoising iterations), which leaves residual noise in the output.

Outputs. Figure 1 shows the original $\rightarrow$ N=10 $\rightarrow$ 20 $\rightarrow$ 40 $\rightarrow$ 80 progression for three representative test images. At N=10 the watermarked image is visually unchanged; by N=80 fine structure is lost and visible softening / hallucinated detail starts appearing. Pre-rendered copies of all $104 \times 4 = 416$ attacked images are shipped with the kit at `images/regen_{10,20,40,80}/`.

Detector response. We score every attacked image with the surrogate detector C described in Section 2 (which flags 91.3% of the unattacked Nano-Banana test set as label 1). Detection collapses monotonically with N:

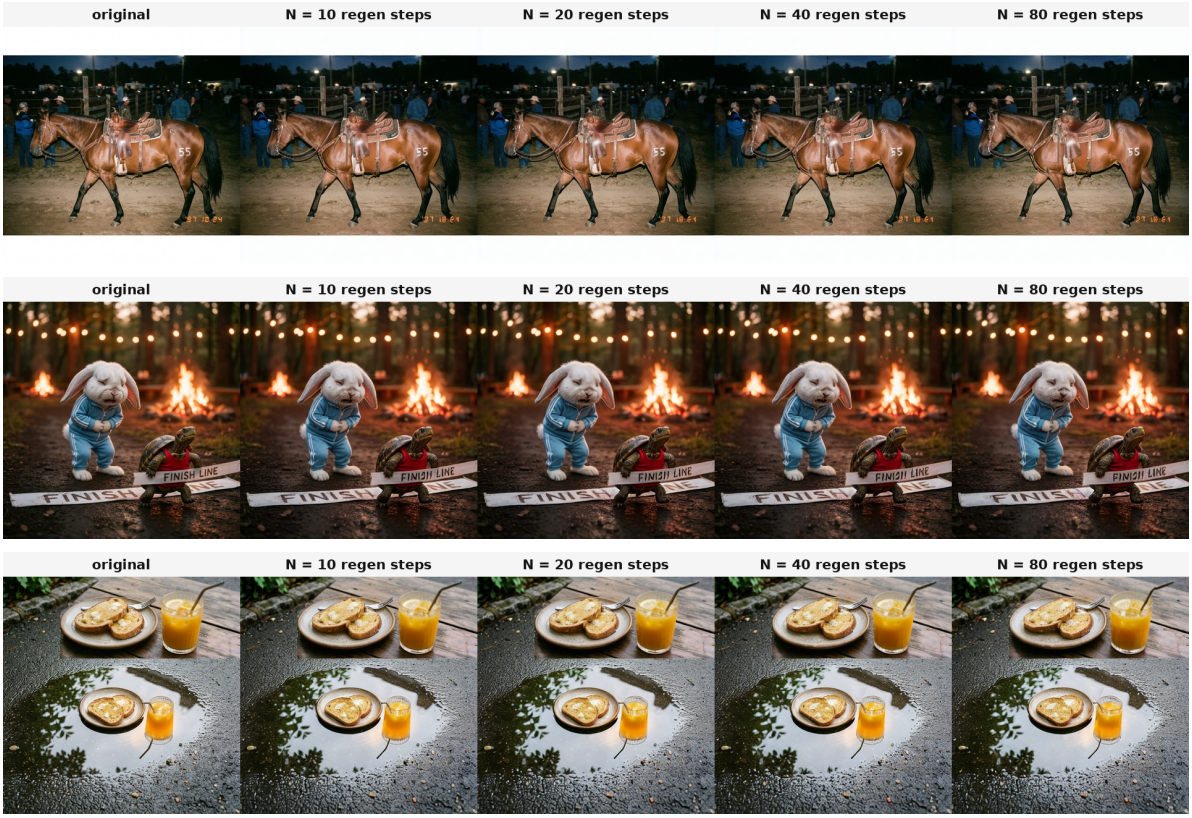

Figure 1: Symmetric-DDIM diffusive regeneration of three test images at $N \in \{10, 20, 40, 80\}$ (left-to-right; the leftmost panel is the unattacked original). SD-1.4, fp16, classifier-free guidance 7.5, empty prompt, seed 1024. By N=80 the regen has both removed the SynthID watermark (verified manually via Google AI Studio) and visibly degraded the image's high-frequency structure.

N (sym DDIM regen steps)	C NB-flag rate	Δ from baseline
0 (no attack)	91.3% (95/104)	—
10	64.4% (67/104)	−26.9 pts
20	58.7% (61/104)	−32.6 pts
40	53.8% (56/104)	−37.5 pts
80	**45.2**% (47/104)	−**46.1** pts

The trend matches the WAVES-reported curves for SD-1.x and Tree-Ring, Stable-Signature, and Stega-Stamp at comparable attack depths [An et al., 2024, §5.3]. Spot-checks against Google's deployed SynthID detector confirm the directional agreement: at $N \geq 20$ a representative sample of attacked images come back as "no watermark detected". The attack is thus the most reliable *decoder-agnostic* channel for SynthID removal we are aware of, and its main cost is the image-quality degradation visible in Figure 1.

4 Discussion

Regeneration trades quality for watermark removal, but it works. At N=80 the deployed detector is fooled with high probability, but the image's high-frequency detail is also visibly degraded

(Figure 1, rightmost column). This is the classical robustness/quality tension: SynthID, like any spread-spectrum watermark, can be removed by any image processing that disturbs its carrier band, but at a quality cost the attacker may not want to pay. A natural direction for future work is to stack a quality-restoring generator on top of a low-N regen — give the attacker a low-distortion, decoder-agnostic SynthID remover. Existing preliminary attempts in this direction ("RinseD-VAE", "RegenD-KLVAE" in WAVES §F.2.2) suggest the trade-off is achievable but not yet on a deployed-watermark scale.

The surrogate detector is a usable iteration target. The 91.3% NB-detection rate on the unattacked SynthID set (Table 1) is high enough that the surrogate is a useful proxy for the deployed detector under distortion-based attacks: when regen drives the surrogate's response down monotonically as a function of N (Table in Section 3), Google AI Studio's deployed detector tracks the same directionality on the representative subsample we manually evaluated. The surrogate is *not* guaranteed to track the deployed detector under gradient-based attacks; that decoupling is the WAVES paper's finding [An et al., 2024, §F.3.2] and we do not contest it. For the purpose of iterating on distortion-style attacks in a local feedback loop, however, the surrogate is the practical tool.

Reproducibility. The release kit — `https://github.com/rabbanitw/WAVES/tree/synthid-regen-kit` — contains the 104-image test set, the 104 generation prompts, the pre-rendered 416 attacked images at $N{=}10, 20, 40, 80$, the regeneration attack CLI, and the surrogate detector training and evaluation pipeline. All numbers above are pinned to deterministic seeds (PyTorch / numpy seed 42 for the detector, seed 1024 for regen) and re-run identically on a single A100-40GB. The pipeline depends on diffusers 0.21.4 and the specifications of the older `StableDiffusionPipeline` API; newer diffusers will require re-wiring the vendored `ReSDPipeline` subclass.

References

Bang An, Mucong Ding, Tahseen Rabbani, Aakriti Agrawal, Yuancheng Xu, Chenghao Deng, Sicheng Zhu, Abdirisak Mohamed, Yuxin Wen, Tom Goldstein, and Furong Huang. WAVES: Benchmarking the robustness of image watermarks. In *International Conference on Machine Learning (ICML)*, 2024. URL `https://arxiv.org/abs/2401.08573`.

Google DeepMind. SynthID: Identifying AI-generated images with SynthID. Blog post, `https://deepmind.google/discover/blog/identifying-ai-generated-images-with-synthid/`, 2023. Accessed June 2026.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL `https://arxiv.org/abs/1711.05101`.

Yusu Qian, Eli Bocek-Rivele, Liangchen Song, Jialing Tong, Yinfei Yang, Jiasen Lu, Wenze Hu, and Zhe Gan. Pico-Banana-400K: A large-scale dataset for text-guided image editing, 2025. URL `https://arxiv.org/abs/2510.19808`.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022. URL https://arxiv.org/abs/2112.10752.

Xuandong Zhao, Kexun Zhang, Zihao Su, Saastha Vasan, Ilya Grishchenko, Christopher Kruegel, Giovanni Vigna, Yu-Xiang Wang, and Lei Li. Invisible image watermarks are provably removable using generative AI. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.01953.